

FT-Behavioural Router

Knowledge vs Behaviour Separation in Adaptive Hybrid Inference

A thesis

submitted by

Vijay Kumar

Dublin, Ireland

vijaykumarjob0701@gmail.com

[LinkedIn](#) [GitHub](#)

September 2026

Version 1.1 (corrected confusion matrices)

This document is written in the usual order used by university repositories, ProQuest, and similar thesis archives: title page, declaration, abstract, contents, lists of figures and tables, chapters, references, and appendices.

Declaration

I declare that this thesis is my own work, except where cited. It has not been submitted, in whole or in part, for any other degree or diploma. Where other people's work is used, it is acknowledged in the text and in the reference list. The experimental numbers reported here are taken from the measured and simulation artefacts named in the chapters; no live cloud bills and no invented answer-quality scores are presented as measured results.

Name: Vijay Kumar

Date: 17 September 2026

Abstract

Hybrid assistants are built using retrieval-augmented generation (RAG), tools, and multiple model sizes, but running a large LLM with retrieval on every query is expensive. In this work, a fine-tuned (FT) *behavioural* router is studied that picks one of four actions—answer as a small model, retrieve and answer as the same small model (FT+RAG), call tools, or escalate to a large LLM—while factual knowledge is kept non-parametric. **No claim is made here of inventing adaptive RAG or LLM cascading**; those areas are well established. The contribution is limited to a unified 4-way behavioural policy, an explicit knowledge≠behaviour separation, and decision criteria for when FT routing is worth the effort compared to strong rule/prompt-style baselines—checked against hardness probes.

Scope. Phase 1 is measured routing classification. Phase 2 is a simulation under a fixed unit cost model (not live API bills).

On in-domain `eval_v2` ($n=133$), FT DistilBERT reached accuracy 0.9699 and MiniLM 0.8872, both above a prompt-rubric simulated baseline (0.7744; deterministic LABELING.md cues—**not** an LLM API). Because near-ceiling DistilBERT scores may indicate that synthetically expanded labels are partly cue-easy, a hard OOD split was also released (`eval_ood`, $n=104$; never used for training) that stresses ambiguity, multi-intent, tools-vs-rag paraphrases, entity-heavy queries, and adversarial keyword traps. On OOD, DistilBERT dropped to 0.8750 ($\Delta \approx -0.095$) and the prompt-rubric collapsed to 0.5000 ($\Delta \approx -0.274$): this gap is reported for honesty, not hidden. In Phase 2 a *simulated* hybrid stack was added under fixed unit costs (FT mean cost 0.287 vs always-RAG+escalate 1.15 on `eval_v2`; large-call rate 0.218 vs 1.0), plus local BM25 hit@1=0.984 and tool-stub schema success 1.0—**not** a cloud bill, **no** live LLM API spend, and **no** invented answer EM/F1.

Keywords: behavioural routing; retrieval-augmented generation; fine-tuning; hybrid inference; knowledge–behaviour separation.

Contents

Declaration	i
Abstract	ii
List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
1 Introduction	1
1.1 Motivation	1
1.2 Phase framing (honest scope)	1
1.3 Originality fence	2
1.4 Research questions	2
1.5 Non-claims	3
1.6 Concrete contributions (v1.1)	3
1.7 Thesis structure	3
2 Related Work	4
2.1 Adaptive retrieval and RAG controllers	4
2.2 Query augmentation routing (QueryRouter)	4
2.3 Model cascading and strong–weak routing	5
2.4 RAG-aware model or corpus selection	5
2.5 Gap and originality fence	5
2.6 Phase 1 positioning relative to neighbours	5
2.7 Prompted routers vs FT routers (related framing)	6
2.8 Behavioural control vs knowledge editing	6
2.9 Evaluation cousins	6
3 Architecture: FT-First Behavioural Control	7
3.1 FT-first control path	7
3.2 Who answers after retrieval (FT+RAG)	8
3.3 Design principles	8
3.4 Rejecting FT-last as the research core	9

3.5	Interface to Phase 1 vs Phase 2	9
3.6	Cost model (Phase 2 simulation)	9
3.7	Training sketch	10
4	Methodology	11
4.1	Action taxonomy and schema	11
4.2	Expanded dataset (v2)	11
4.3	Worked labeling examples	12
4.4	Router models and baselines	12
4.5	Metrics and uncertainty	13
4.6	Phase 2 simulation protocol	13
4.7	Reproducibility	14
5	Experiments	15
5.1	Protocol overview	15
5.2	Hypotheses (summary)	15
5.3	Routing classification results (v2, Phase 1)	16
5.4	Error analysis from confusion matrices	16
5.5	Paraphrase consistency	17
5.6	Hard OOD generalisation (honesty probe)	18
5.7	Phase 2 hybrid stack simulation	18
5.8	Experimental environment	19
5.9	Remaining ablations (future)	20
6	Discussion	21
6.1	Positioning honesty	21
6.2	Reading the Phase 1 numbers	21
6.3	When is FT worth it?	22
6.4	Threats to validity	22
6.5	Failure modes	23
6.6	Limitations	23
6.7	Ethics	24
6.8	Implications of the Phase 2 simulation	24
7	Conclusion	25
A	Dataset, OOD, and Camera-Ready Checklist	27
A.1	Dataset statistics (v2 + OOD)	27
A.2	Labeling decision order (summary)	27
A.3	Hard OOD construction (summary)	28
A.4	OOD confusion matrices (corrected label order)	28
A.5	Paraphrase pairs	29
A.6	Result artefact map	29
A.7	Honesty checklist for readers	29

A.8 Camera-ready checklist (v1.0)	30
A.9 Erratum: confusion matrix label ordering (v1.1)	30
References	32

List of Figures

3.1	FT-first hybrid behavioural control	7
-----	---	---

List of Tables

2.1	Capability coverage compared with this work	5
3.1	Execution paths from the 4-way labels	8
5.1	Phase 1 routing metrics on eval_v2	16
5.2	Phase 1 stratified 5-fold	16
5.3	In-domain vs hard OOD routing metrics	18
5.4	Phase 2 simulated cost on eval_v2	19
5.5	Phase 2 simulated cost on eval_ood	19
A.1	Split sizes and primary-action counts (validated).	27
A.2	Domain counts on train_v2 / eval_v2 / eval_ood.	27

List of Abbreviations

Abbreviation	Meaning
BM25	Ranking function used for local retrieval
CI	Confidence interval
EM	Exact match
FT	Fine-tuned / fine-tuning
LLM	Large language model
OOD	Out of distribution
RAG	Retrieval-augmented generation
SFT	Supervised fine-tuning

Chapter 1

Introduction

1.1 Motivation

Modern assistant systems are built using retrieval-augmented generation (RAG) [Lewis et al., 2020], tool calling, and multiple model sizes behind a single interface. Running a large language model (LLM) with retrieval on every query is expensive in terms of both cost and compute; answering everything with a small parametric model fails on freshness, long-tail facts, and hard multi-step reasoning. The *behavioural* decision—which capability should be used for a given query—is therefore the key lever for controlling cost, latency, and reliability.

Several pieces of this decision have been explored in prior work: when to retrieve [Jeong et al., 2024, Tan et al., 2024, Asai et al., 2023], which input augmentation to apply for a single LLM [Chen et al., 2025], and which model size to call [Ong et al., 2025, Ding et al., 2024, Chen et al., 2023, Madaan et al., 2024]. What has received less attention is a *unified* behavioural policy that jointly picks among answering as a small model, answering as the same small model with retrieval (FT+RAG), calling tools, and escalating to a large LLM, while factual knowledge is kept non-parametric.

1.2 Phase framing (honest scope)

This document is a **workshop / short-paper oriented thesis** (v1.1, corrected confusion matrices) organised in two phases:

Phase 1 (measured routing). An expanded *routing classification* study with a hard OOD honesty probe. Workplace-style queries were labelled with a mutually exclusive four-way primary action. Majority, keyword, prompt-rubric simulated, and fine-tuned encoder classifiers were evaluated on in-domain `eval_v2` ($n=133$; train $n=526$) and on held-out `eval_ood` ($n=104$; never used for training). Metrics reported are accuracy, macro-F1, confusion matrices, bootstrap/ k -fold (in-domain), and in-domain→OOD gaps—**only measured numbers** appear in Phase 1 result tables.

Phase 2 (simulation). A fixed unit cost/latency model, local BM25 mini-RAG hit@ k , and tool-stub schema checks were wired to gold and DistilBERT routes. System

tables in Chapter 5 are filled **only** from `results/phase2/*.json` and labelled as simulation—not cloud bills, not live LLM API spend, and not live answer quality.

By framing Phase 1 as measured routing and Phase 2 as simulation, overclaiming of system-level product gains from a routing classifier alone is avoided.

1.3 Originality fence

No claim is made here of inventing adaptive RAG or LLM cascading. Adaptive retrieval and complexity-based routing were established by Adaptive-RAG [Jeong et al., 2024], SlimPLM [Tan et al., 2024], Self-RAG [Asai et al., 2023], and related systems. Strong-weak and cascade routing were established by RouteLLM [Ong et al., 2025], HybridLLM [Ding et al., 2024], FrugalGPT [Chen et al., 2023], and AutoMix [Madaan et al., 2024]. QueryRouter [Chen et al., 2025] fine-tunes a lightweight router over augmentation paths for a single LLM.

Contribution niche. A thinner open region is targeted:

1. a **unified 4-way behavioural policy** $a \in \{\text{answer_small}, \text{rag}, \text{tools}, \text{escalate_large}\}$, where `rag` is the small FT model plus retrieval (FT+RAG), and `large+RAG` is `escalate_large` with `needs_rag=true` (Section 3.2);
2. an explicit **knowledge $\neq$ behaviour** separation (facts are kept in the corpus/APIs; only the routing policy is fine-tuned);
3. empirical **when-is-FT-worth-it** decision criteria versus strong rule/prompt-style baselines under measured Phase 1 deltas (live prompted large routers and natural domain shift remain future measured work beyond the Phase 2 simulation).

1.4 Research questions

Can a lightweight fine-tuned behavioural model improve the efficiency and reliability of a RAG-augmented LLM by dynamically deciding when to (i) answer directly as the small model, (ii) retrieve external knowledge and still answer as the small model (FT+RAG), (iii) invoke tools, or (iv) delegate reasoning to a larger LLM?

Secondary questions for the full agenda: when does FT beat a strong prompted router under shift? Does knowledge/behaviour separation improve freshness without hurting quality?

Phase 1 addresses a necessary first step: *can a small encoder learn the 4-way labelling policy from expanded data better than simple and strong rule baselines, with quantified uncertainty?* In Phase 2 a simulated cost/latency layer is added under fixed unit rates; live answer quality remains out of scope.

1.5 Non-claims

The following are explicitly not claimed: inventing adaptive retrieval, inventing cascading, replacing RAG with small-model memorisation, or that fine-tuning always beats prompting for routing. The prompt-rubric baseline is not presented as an LLM-API prompted router. Live cloud \$/query savings, live LLM API timings, or end-to-end answer EM/F1 are not claimed: Phase 2 reports *simulated* route-composition costs only. High in-domain DistilBERT accuracy is not taken to mean near-solved robust routing—the OOD probe was created to temper that reading.

1.6 Concrete contributions (v1.1)

- Expanded, schema-validated routing labels ($n=526/133$) with documented labeling rules and paraphrase pairs.
- A hard OOD eval split ($n=104$) and a measured in-domain vs OOD gap table (models were not trained on OOD).
- A measured baseline ladder including an honest prompt-rubric simulation.
- FT MiniLM and DistilBERT results with bootstrap/ k -fold (in-domain) and OOD generalisation; claims are tempered where DistilBERT’s in-domain ceiling drops under hardness.
- Confusion-matrix error analysis and explicit when-is-FT-worth-it decision criteria.
- Phase 2 *simulation* system tables sourced from `results/phase2/` JSON only, plus a completed camera-ready checklist in the appendix.

1.7 Thesis structure

In Chapter 2, related work is positioned under the originality fence. In Chapter 3, the FT-first architecture is described and FT-last is rejected as the research core. Chapter 4 covers schema, v2 data, models, baselines, and metrics. In Chapter 5, Phase 1 measured routing results (including OOD gaps and error analysis) and Phase 2 simulated system cost tables are reported. Chapters 6–7 cover threats to validity, when FT is worth it, ethics, and next steps. Appendix A holds dataset counts, OOD matrices, and the honesty checklist.

Chapter 2

Related Work

In this chapter, novelty is fenced relative to established lines of adaptive retrieval, query augmentation routing, and strong-weak / cascade model selection. **No claim is made** of inventing adaptive RAG or LLM cascading. The contribution is limited to a unified *4-way behavioural* policy {`answer_small`, `rag` (FT+RAG), `tools`, `escalate_large`}, an explicit knowledge≠behaviour training target, and an empirical when-is-FT-worth-it study under domain shift. FT-first (routing/control) is the architecture under study; FT-last (post-generation rewrite) is kept out of scope.

2.1 Adaptive retrieval and RAG controllers

Adaptive-RAG [Jeong et al., 2024] trains a classifier over query complexity to choose among no-retrieval, single-step retrieval, and multi-step RAG for a single LLM stack. In SlimPLM [Tan et al., 2024], a slim proxy model is used to judge *when* and *what* to retrieve for a large LLM, reducing unnecessary retrieval cost. In Self-RAG [Asai et al., 2023], retrieve / critique decisions are embedded inside one generation model via reflection tokens. CRAG [Yan et al., 2024] evaluates and corrects retrieved evidence (with optional web fallback). In Auto-RAG [Yu et al., 2024], retrieve-reason loops are iterated autonomously inside an LLM. Together these systems cover *when/what to retrieve* and corrective RAG quality well. However, tools and escalate-to-large are not jointly treated as first-class peers of a standalone-small answering path and a small-model FT+RAG path, nor is an explicit knowledge≠behaviour behavioural policy trained over the 4-way action space used here.

2.2 Query augmentation routing (QueryRouter)

QueryRouter [Chen et al., 2025] (Findings of NAACL 2025) addresses “when and how to augment the input” by routing each query to an augmentation path (e.g. chain-of-thought, rewrite, expansion, retrieval, or none) before answering with one LLM. It is a close neighbour on *fine-tuned lightweight routing* for accuracy-efficiency, but the decision space is *input augmentation for a single model*, not a hybrid behavioural controller where answering as a small model, answering as the small model with RAG (FT+RAG), calling tools, or escalating to a larger model are all peer actions.

2.3 Model cascading and strong-weak routing

In FrugalGPT [Chen et al., 2023], cascades over API models with early stopping for cost are learned. HybridLLM [Ding et al., 2024] routes between small and large models using quality-aware signals. In RouteLLM [Ong et al., 2025], routers are trained from human preference data for strong-weak selection and out-of-domain generalisation is studied. AutoMix [Madaan et al., 2024] mixes models via verification before escalation. In SATER [Shen et al., 2025], token-efficient routing and cascading with self-aware rejection is improved. These lines establish cost-quality model selection and cascading; RAG and tools are typically left out of the same primary action space. In this work, cascading is adopted as prior art and retrieval/tools are added alongside standalone-small and escalate as one policy.

2.4 RAG-aware model or corpus selection

In RAGRouter [Zhang et al., 2025], selection is done among multiple RAG-enabled LLMs given retrieved context. UniversalRAG [Yeo et al., 2025] routes across modality and granularity corpora. RouterBench [Hu et al., 2024] benchmarks multi-LLM routing systems. These pick *which knowledge path or which model under RAG*, not the 4-way behavioural space {`answer_small`, `rag` (small FT + retrieval), `tools`, `escalate_large`} studied here.

2.5 Gap and originality fence

Table 2.1: Capability coverage vs. this work (✓ present, ∼ partial, — absent). The joint combination is claimed, not adaptive RAG or cascading alone.

System	RAG	Tools	Small-standalone	Escalate	Know. ≠ beh.
Adaptive-RAG	✓	—	∼	—	—
QueryRouter	✓	—	∼	—	—
SlimPLM	✓	—	∼	∼	∼
RouteLLM / HybridLLM	—	—	✓	✓	—
FrugalGPT / AutoMix	—	—	✓	✓	—
Self-RAG / Auto-RAG	✓	—	—	—	—
This work	✓	✓	✓	✓	✓

Prior work covers (a) when to retrieve, (b) which augmentation for one LLM, (c) which model size, and (d) which corpus/model under RAG (Table 2.1). What remains thin is a single behavioural policy over {`answer_small`, `rag` (FT+RAG), `tools`, `escalate_large`} with knowledge≠behaviour training and a clear FT-vs-prompt worth-it study under shift. That niche is adopted here without claiming to invent adaptive RAG or cascading; FT-last presentation polish is not the research core.

2.6 Phase 1 positioning relative to neighbours

In Phase 1, Adaptive-RAG complexity classes or RouteLLM preference routers are not re-implemented. Instead, it is measured whether a small FT classifier can learn the labelled

4-way behavioural policy on expanded v2 data ($n=526/133$) relative to majority, keyword, and a *prompt-rubric simulated* baseline that encodes LABELING.md without an LLM API. Comparisons to true prompted large routers, Adaptive-RAG-like controllers, and strong-weak systems remain *future measured work* beyond the Phase 2 fixed-unit simulation, so that routing-label accuracy is not mixed up with live retrieval or cascade quality. This keeps the originality fence intact: adaptive retrieval and cascading are adopted as prior art and only the joint behavioural niche claimed in Chapter 1 is evaluated.

2.7 Prompted routers vs FT routers (related framing)

A recurring question in routing literature is whether a fine-tuned small router beats a carefully prompted large model under distribution shift [Ong et al., 2025, Chen et al., 2025]. Phase 1 contributes a *lower bound* version of that comparison: FT vs a deterministic rubric that approximates the prompt an LLM would receive. Beating the rubric is a necessary condition for FT to be interesting; beating a true LLM prompt under natural shift remains future measured work and is part of the claimed when-is-FT-worth-it niche—not a claim that FT always wins.

2.8 Behavioural control vs knowledge editing

A parallel line of research edits or fine-tunes model weights to store facts. That path is deliberately separated from behavioural routing in this work: encoding company SLAs or yesterday’s incident text into weights creates problems with freshness and auditability. RAG and tools are kept as the knowledge plane; FT is used only for the control plane. This approach aligns with corrective and adaptive RAG systems that keep knowledge external [Yan et al., 2024, Asai et al., 2023, Jeong et al., 2024], while differing in the 4-way behavioural action set.

2.9 Evaluation cousins

In Adaptive-RAG-style evaluations, routing quality plus downstream answer utility are emphasised when retrieval depth changes [Jeong et al., 2024]. In RouteLLM-style evaluations, quality–cost curves are emphasised when hard queries escalate [Ong et al., 2025]. Phase 1 metrics here cover the routing-label half of those cousins; Phase 2 adds a *simulated* cost/latency half under fixed unit rates. Live answer utility and cloud bills remain unmeasured before product-level system claims can be made.

Chapter 3

Architecture: FT-First Behavioural Control

3.1 FT-first control path

The fine-tuned small model is placed as the **first** decision layer in the hybrid stack. Given a user query q , a behavioural router R_θ predicts a primary action

$$a = R_\theta(q) \in \{\text{answer_small}, \text{rag}, \text{tools}, \text{escalate_large}\}.$$

Downstream modules (small responder, retriever, tool runtime, large LLM) are executed only when selected.

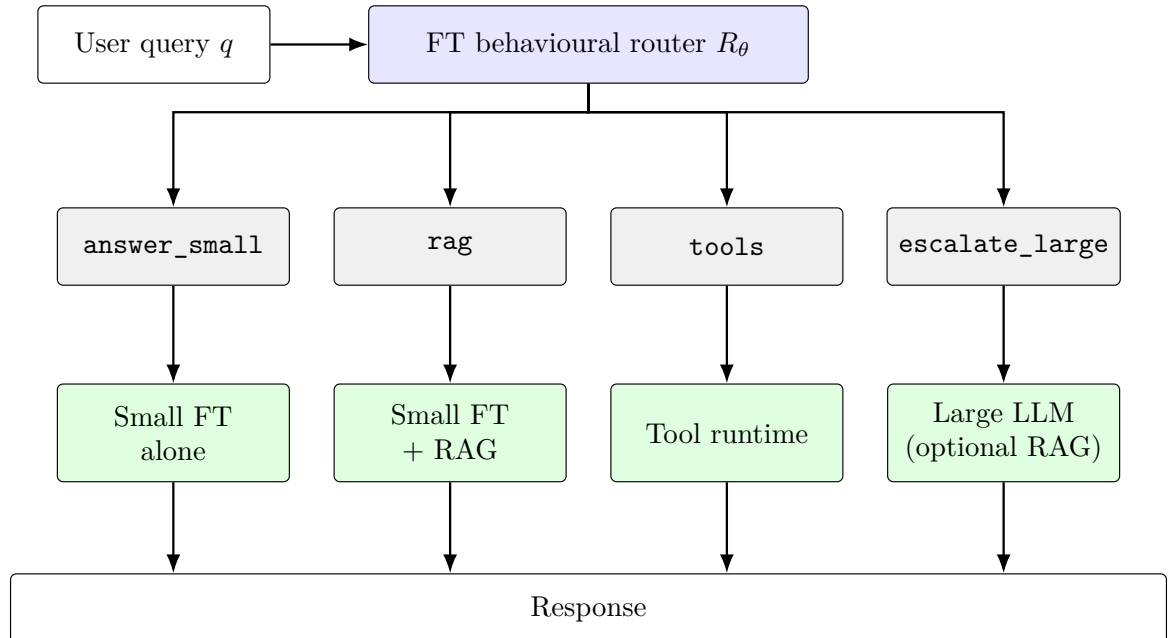


Figure 3.1: FT-first hybrid behavioural control. The path is chosen by the router *before* expensive generation is started. **answer_small** is the small FT model alone; **rag** is the same small FT model with retrieved documents (FT+RAG). The large LLM is used only on **escalate_large** (with optional RAG via the **needs_rag** flag). This diagram is not claimed to invent adaptive RAG or cascading—only the joint 4-way behavioural policy is under study.

The path is controlled first by the router. On the **rag** path, retrieved documents are given to the *small* FT responder—not to the large LLM. The large LLM receives optional RAG/tool context only when the primary action is **escalate_large** (and the matching aux flags are set). RAG and tools are not post-processing after a full answer has already been generated.

3.2 Who answers after retrieval (FT+RAG)

The 4-way primary label space is kept (Phase 1 numbers are not retrained). Five *execution* paths are obtained from those four labels plus the **needs_rag** flag:

Table 3.1: Execution paths. Phase 1 still classifies the 4-way **primary_action** only. FT+RAG is the **rag** class: the small model answers using retrieved docs. Large+RAG is **escalate_large** with **needs_rag=true**.

Execution path	Primary label	Who answers	Retrieval
FT alone	answer_small	small FT model	no
FT + RAG	rag	small FT model	yes
Tools	tools	tool runtime	optional
Large alone	escalate_large	large LLM	no
Large + RAG	escalate_large + needs_rag	large LLM	yes

rag is used when the missing piece is a fact, not hard reasoning: for example a company SLA, yesterday’s incident summary, or a runbook lookup. The small FT model is kept frozen for behaviour (tone, format, refusal, how to use context); the corpus can be updated without retraining that model. If the query also needs multi-hop analysis, **escalate_large** with **needs_rag=true** is used instead, so the large model sees the same retrieved docs.

Honesty: Phase 1 measures whether the router can pick the **rag** label, not whether the small model then writes a correct answer from those docs. Phase 2 prices **rag** as retrieval *without* a large-LLM call (**large_call=false** in the unit cost model), which matches this FT+RAG path.

3.3 Design principles

- **Knowledge $\neq$ behaviour.** The small model is not fine-tuned to memorise long-tail or time-sensitive facts. Facts are kept in the corpus/API; fine-tuning is used for routing and simple behavioural replies only.
- **4-way action space.** The primary action *a* is mutually exclusive; secondary flags (**needs_rag**, **needs_tools**, **reasoning_level**) are used to capture composites without making the label space too large. The **rag** primary is the FT+RAG path (Table 3.1).
- **Small may answer alone or with RAG.** Greetings, format transforms, and in-distribution procedural asks go to **answer_small**. Company/time-sensitive lookups go to **rag**: the same small FT model, plus retrieved documents.

- **Large owns hard reasoning.** Multi-hop, ambiguous, safety-critical, or low-confidence cases are escalated. If facts are also missing, `needs_rag=true` is set (large + RAG), not the small FT+RAG path.
- **Freshness via RAG/tools.** Time-sensitive facts are never expected to come from small-model parameters. Updating the corpus does not require retraining the FT responder.

3.4 Rejecting FT-last as the research core

An alternative approach is *FT-last*: generate with a large LLM (possibly with RAG/tools), then fine-tune a small model to rewrite tone or format. FT-last is rejected as the **core** architecture for this project:

- A missed retrieval, tool, or escalate decision cannot be recovered after the fact.
- Little cost saving is achieved once the expensive path has already been run.
- Rewriting a full LLM answer with a small FT model risks factual drift.

FT-last may be used later as optional polish for tone or compliance; it is **not** the behavioural control layer studied here. Behaviour here means *system decisions*, not cosmetic rewriting.

3.5 Interface to Phase 1 vs Phase 2

In Phase 1, R_θ is implemented as a sequence classifier and its agreement with v2 oracle labels is measured, including uncertainty estimates. In Phase 2 (run as a *simulation*), predicted and gold actions are wired into a fixed unit cost/latency model, a local BM25 mini-corpus, and deterministic tool stubs (`configs/phase2_cost_model.yaml`, `data/phase2/`, `src/fta_router/phase2/`). **Honesty:** Phase 2 reports simulated \$/query and latency under documented unit rates—not a cloud bill, not live LLM API timings, and not answer EM/F1 or Arena scores [Yan et al., 2024, Asai et al., 2023].

3.6 Cost model (Phase 2 simulation)

Each action is mapped to a documented unit cost and latency in `configs/phase2_cost_model.yaml` (`answer_small`, `rag` as FT+RAG, `tools`, `escalate_large`, plus `escalate+RAG/tools` combinations). The `rag` profile has `large_call=false`: retrieval plus the small FT responder, not a large-LLM call. Cascade baselines force `always_escalate` or `always_rag_escalate`; the FT policy uses DistilBERT primary predictions (secondary flags are derived from the predicted primary only—no gold leakage). All cascade comparisons use the *same* arithmetic; relative costs are ratios of means under that model.

3.7 Training sketch

Oracle (or preference) labels for the best action given query and cost/quality constraints are collected; a small encoder (MiniLM, DistilBERT) or later a 0.5B–3B generative router with constrained decoding is supervised fine-tuned. Optionally, distillation from a prompted large router can be done, then it can be tested whether FT retains an edge under distribution shift (Chapter 5: Phase 1 OOD precursor; live prompted routers remain future measured work).

Chapter 4

Methodology

4.1 Action taxonomy and schema

Each example carries a mutually exclusive `primary_action` in `{answer_small, rag, tools, escalate_large}` plus auxiliary fields `needs_rag`, `needs_tools`, `reasoning_level`, `rationale`, and `domain`. Invariants are enforced by validation: `primary_action=rag  $\Rightarrow$  needs_rag=true`; `primary_action=tools  $\Rightarrow$  needs_tools=true`. Full field definitions are kept in `data/routing/schema.md` (schema version 1.0). The annotator decision order is documented in `data/routing/LABELING.md`.

Interpretation. `answer_small` covers parametric or format-only asks (and mild refusal routing) answered by the small FT model with no retrieval; `rag` is the **FT+RAG** path: missing facts (especially company/time-sensitive) are retrieved, then the *same* small FT model writes the answer from those documents; `tools` covers side-effect or live API intents; `escalate_large` covers hard reasoning/planning where the bottleneck is cognition rather than a single missing document. When facts *and* deep analysis are both needed, `escalate_large` with `needs_rag=true` is preferred (large + RAG) rather than collapsing the policy into small-model retrieval-only.

4.2 Expanded dataset (v2)

The repository ships seed JSONL and expanded v2 splits:

- `train_v2.jsonl`: $n=526$ (the original $n=128$ seed is included as prefix).
- `eval_v2.jsonl`: $n=133$ (the original $n=38$ seed is included as prefix).
- `paraphrase_pairs_v2.json`: 24 paraphrase pairs with identical gold `primary_action`.

Class balance (eval_v2). `answer_small` 41, `rag` 31, `tools` 31, `escalate_large` 30—near-balanced four-way, which is why both accuracy and macro-F1 are reported.

Domains. `general_knowledge`, `company_current`, `tool_action`, `multi_hop_planning`, `ambiguous`, and `safety_mild`.

Labeling quality controls. The annotator decision order is `tools` → `rag` → `escalate` → `answer_small` based on intent (`LABELING.md`); invariants are validated by `scripts/validate_dataset.py`; paraphrase pairs are reserved for consistency analysis; mild safety is never labelled as `tools`.

Hard OOD probe (`eval_ood`). A separate hardness split ($n=104$) was created to stress ambiguity, multi-intent, tools-vs-rag paraphrases, entity-heavy company queries, and adversarial keyword traps (documented in `LABELING.md`). Queries are disjoint from `train_v2/eval_v2`. **No model was trained or selected on OOD**; FT early-stop uses `eval_v2` only. The probe was created because DistilBERT’s near-ceiling in-domain accuracy may indicate that synthetically expanded v2 labels are partly cue-aligned / too easy—OOD serves as the honesty check.

Limitations of labels. Labels are synthetic workplace-style constructions with human-written rationales, not production logs. Inter-annotator agreement has not been measured yet. High in-domain FT accuracy partly means “fits the labelling policy,” which is necessary but not enough for product quality—especially if templates are regular. Phase 1 therefore supports conclusions about learnability of the labelling policy, comparative baseline strength, and hardness-gap honesty, not deployment claims.

4.3 Worked labeling examples

- “What is a mutex?” → `answer_small` (parametric definition).
- “What is our SLA for the payments API?” → `rag` (FT+RAG: company fact retrieved, small FT model answers).
- “Create a Jira ticket titled ...” → `tools` (side effect).
- “Analyse yesterday’s outage and propose a multi-week plan” → `escalate_large` with `needs_rag=true` (large + RAG: facts plus hard reasoning).
- “Summarise yesterday’s outage and file a Jira” → `escalate_large` with both aux flags true.
- Mild policy-sensitive asks (e.g. deceptive credential harvesting) → `answer_small` refusal path, never `tools`.

4.4 Router models and baselines

Non-neural baselines.

- **Majority:** the most frequent training `primary_action` is always predicted.
- **Keyword heuristic:** ordered regex rules for `tools`, `escalate`, and `RAG` cues are applied; default is `answer_small`.

- **Prompt-rubric simulated:** weighted cue scoring based on LABELING.md is used, with tie-break order tools > escalate_large > rag > answer_small as implemented in src/fta_router/baselines.py, plus an explicit safety override to answer_small. **Honesty:** this is *not* an LLM API call and *not* a neural zero-shot model; it is a strong deterministic stand-in for a prompted rubric when no API key is available.

Neural FT routers. Sequence classification is done with MiniLM (sentence-transformers/all-MiniLM) and DistilBERT (distilbert-base-uncased) on query text $\rightarrow$ primary_action. Default hyperparameters: 5 epochs, batch size 16, lr 2×10^{-5} , seed 42, max length 128 (configs/router_minilm_v2.yaml, configs/router_distilbert_v2.yaml).

Deferred baselines (beyond Phase 2 simulation). True prompted large 4-way router via LLM API; Adaptive-RAG-like complexity classifiers [Jeong et al., 2024]; strong-weak-only routers [Ong et al., 2025, Ding et al., 2024]; cascade/AutoMix-like systems [Madaan et al., 2024, Chen et al., 2023]. Phase 2 simulation compares cascade *cost* policies under fixed unit rates; these live routers are not run.

4.5 Metrics and uncertainty

- **Point metrics:** accuracy, macro-F1, confusion matrix over the four actions (fixed label order).
- **Bootstrap CI:** 1000 resamples of eval predictions from a single trained model; mean $\pm$ std and 95% percentile intervals are reported.
- **Stratified k -fold:** $k=5$ on combined train+eval (3 epochs/fold); mean $\pm$ std across folds is reported as a train/eval-split sensitivity check.
- **Paraphrase agreement:** the fraction of pairs where predicted labels match on A and B is measured.
- **Phase 2 system (simulation, executed):** mean cost/query, large-LLM call rate, and latency under the fixed unit model; local RAG hit@ k ; tool-stub schema success. Downstream answer quality, freshness, and FT-vs-true-prompt Δ under natural shift remain future measured work.

4.6 Phase 2 simulation protocol

Phase 2 does **not** call live LLM answer APIs. Four artefacts are measured under an explicit honesty fence:

1. **Route $\rightarrow$ cost/latency model.** Fixed unit costs and latencies per action and documented combinations (configs/phase2_cost_model.yaml). These are applied to gold labels and DistilBERT predicted routes on eval_v2 and eval_ood. Mean cost, large-call rate, and p50/p95 latency *under the model* are reported. This is a simulation, not a cloud bill.

2. **Local mini-RAG.** A tiny company-ish markdown corpus (`data/phase2/corpus/`) with BM25 retrieval is used; hand-built qrels; $\text{hit}@k$ for queries labelled `needs_rag` / `primary rag`.
3. **Tool stubs.** Deterministic fake `jira_create` / `github_comment` with schema validation success rate when `primary=tools`.
4. **Cascade comparison.** `always_escalate` vs `always_rag_escalate` vs FT-routed policy under the same cost model (real arithmetic from routes).

Not allowed: inventing EM/F1 from GPT answers, fake Arena scores, or pretend API runs.

4.7 Reproducibility

```
source .venv/bin/activate
pip install -e ".[dev]" ".[train]"
python scripts/validate_dataset.py \
    --train data/routing/train_v2.jsonl --eval data/routing/eval_v2.jsonl
python scripts/train_router_local.py --baseline majority \
    --train data/routing/train_v2.jsonl --eval data/routing/eval_v2.jsonl --results-dir re
python scripts/train_router_local.py --baseline keyword \
    --train data/routing/train_v2.jsonl --eval data/routing/eval_v2.jsonl --results-dir re
python scripts/train_router_local.py --baseline prompt_rubric \
    --train data/routing/train_v2.jsonl --eval data/routing/eval_v2.jsonl --results-dir re
python scripts/eval_bootstrap_ci.py --model sentence-transformers/all-MiniLM-L6-v2 \
    --output results/v2/minilm_v2_ci.json --kfold 5
python scripts/eval_bootstrap_ci.py --model distilbert-base-uncased \
    --output results/v2/distilbert_v2_ci.json --kfold 5
python scripts/eval_ood_suite.py
python scripts/run_phase2_sim.py
```

All Phase 1 numbers in Chapter 5 are taken from measured JSON under `results/v2/` and `results/ood/` regenerated by these commands. Phase 2 tables use only `results/phase2/*.json` (`scripts/run_phase2_sim.py`).

Chapter 5

Experiments

5.1 Protocol overview

Phase 1 (executed). Supervised primary-action classification was run on v2 JSONL splits ($n=526$ train, $n=133$ in-domain eval) plus hard OOD ($n=104$; `results/ood/`). Methods tested: majority, keyword heuristic, prompt-rubric simulated, FT MiniLM, FT DistilBERT. FT models were trained on `train_v2` only (early-stop monitor: `eval_v2`); **OOD was never used for training or selection**. Uncertainty (in-domain) was estimated using bootstrap ($B=1000$) and stratified 5-fold on combined data (3 epochs/fold). All numbers below are measured; none are invented.

Phase 2 (executed as simulation). A fixed unit cost/latency model + local BM25 mini-RAG + tool stubs were used (`scripts/run_phase2_sim.py` → `results/phase2/`). Cascade comparison: always-escalate vs always-RAG+escalate vs FT DistilBERT routes (and gold oracle) under the *same* model. **Not** executed: live LLM answer APIs, EM/F1, Arena scores. True prompted large 4-way routers and Adaptive-RAG-like controllers remain future measured work [Jeong et al., 2024, Ong et al., 2025, Ding et al., 2024, Madaan et al., 2024].

5.2 Hypotheses (summary)

H1–H3: cost, latency, and quality vs always-RAG+large and strong-weak-only (Phase 2 simulation covers route-composition cost/latency under fixed unit rates; live answer quality remains future work). H4: freshness under knowledge≠behaviour vs knowledge-FT anti-pattern (future measured work). H5: route consistency under paraphrase (Phase 1: gold pairs + measured model agreement). H6: when FT beats prompt under shift (Phase 1 precursor: FT vs prompt-rubric on in-domain v2 *and* hard OOD; true LLM prompt + natural shift: future measured work).

Table 5.1: Phase 1 (measured routing): primary-action metrics on `eval_v2` ($n=133$). Values are measured. Neural rows: seed 42, 5 epochs; bootstrap mean $\pm$ std from $B=1000$ resamples of eval predictions.

Method	Accuracy	Macro-F1	Boot. acc. (mean $\pm$ std)	Notes
Majority baseline	0.3083	0.1178	—	Always train-mode label
Keyword heuristic	0.5789	0.5302	—	Regex TOOL/RAG/ESCALATE
Prompt-rubric sim.	0.7744	0.7658	—	Not an LLM API
FT MiniLM (ours)	0.8872	0.8829	0.8856 $\pm$ 0.0270	<code>minilm_v2.json</code>
FT DistilBERT (ours)	0.9699	0.9692	0.9692 $\pm$ 0.0152	<code>distilbert_v2.json</code>

Table 5.2: Phase 1 (measured): stratified 5-fold on combined train+eval ($n=659$), 3 epochs/fold (mean $\pm$ std).

Method	Accuracy (mean $\pm$ std)	Macro-F1 (mean $\pm$ std)
FT MiniLM	0.7664 $\pm$ 0.0453	0.7398 $\pm$ 0.0506
FT DistilBERT	0.9105 $\pm$ 0.0217	0.9083 $\pm$ 0.0220

5.3 Routing classification results (v2, Phase 1)

Reading the tables (tempered). Majority (≈ 0.31) is the chance floor under mild imbalance. Keyword (0.58 / 0.53) recovers many `tools` cues but leaks `escalate` into `answer_small`. The prompt-rubric simulated baseline (0.77 / 0.77) is a *strong* non-neural reference for Phase 1: FT must beat it to justify the training cost on this labelling policy. MiniLM clears that bar on the held-out split (0.887); DistilBERT is higher still (0.970). k -fold means are lower than the single held-out point estimates (especially MiniLM), which is expected when folds mix the original seed/eval boundary and use fewer epochs—both numbers are reported without preferring the more flattering one alone. **Caveat:** DistilBERT’s near-ceiling in-domain score is *suspicious* relative to label simplicity; the hard OOD drop in Section 5.6 is reported as the honesty check.

5.4 Error analysis from confusion matrices

Label order throughout: `answer_small`, `rag`, `tools`, `escalate_large` (rows = true, columns = predicted).

Majority. All 133 predictions are `answer_small`: first column [41, 31, 31, 30].

Keyword (true $\times$ pred).

```

answer_small:    37, 2, 2, 0
rag:             18,10, 3, 0
tools:           6, 0,25, 0
escalate_large:  20, 3, 2, 5

```

Dominant failure: `escalate` $\rightarrow$ `answer_small` (20) and `rag` $\rightarrow$ `answer_small` (18) when lexical `escalate`/`RAG` cues are weak.

Prompt-rubric simulated.

answer_small:	38, 1, 2, 0
rag:	9,17, 3, 2
tools:	3, 0,28, 0
escalate_large:	5, 3, 2,20

Tools recall is strong (28/31). Remaining errors are concentrated on rag without overt “our/yesterday” cues and escalate examples that lack explicit plan/tradeoff vocabulary.

MiniLM.

answer_small:	37, 2, 1, 1
rag:	2,23, 2, 4
tools:	0, 0,29, 2
escalate_large:	0, 0, 1,29

Tools and escalate are largely resolved; rag is the hardest class, confused with escalate_large (4) and scattered into answer_small (2) and tools (2)—company/time-sensitive queries whose language overlaps with analytical planning or procedural asks.

DistilBERT.

answer_small:	40, 0, 1, 0
rag:	0,29, 2, 0
tools:	0, 0,31, 0
escalate_large:	0, 0, 1,29

Near-diagonal; tools achieves perfect recall (31/31). The few residual errors are tools over-predictions (answer_small→tools 1; rag→tools 2; escalate→tools 1), consistent with a slight tools bias on queries whose intent overlaps with API/tool vocabulary.

Qualitative takeaway. As baselines get stronger from keyword→rubric→FT, the residual confusion mass shifts from “miss escalate/rag entirely” toward mild tools over-prediction on queries whose intent overlaps with API vocabulary—a different operational risk (unnecessary tool calls on retrieval or reasoning queries) than under-escalation (missed reasoning needs).

5.5 Paraphrase consistency

Twenty-four gold paraphrase pairs share `primary_action` by construction (agreement 1.0). Measured pair agreement (`results/ood/paraphrase_consistency.json`, same 24 pairs): majority 1.000 (trivial constant), keyword 0.750, prompt-rubric 0.917, FT MiniLM 0.875, FT DistilBERT 0.917. This supports H5 at labelling and model layers for paraphrase invariance; it does *not* imply OOD robustness (next section).

Table 5.3: Phase 1 (measured routing): in-domain (`eval_v2`, $n=133$) vs hard OOD (`eval_ood`, $n=104$). FT was trained on `train_v2` only; OOD was never used for training/selection. Values from `results/v2/` and `results/ood/`.

Method	Acc (ID)	Acc (OOD)	Δ Acc	Macro-F1 (ID)	Macro-F1 (OOD)	Δ F1
Majority	0.3083	0.2596	-0.0487	0.1178	0.1031	-0.0148
Keyword	0.5789	0.3654	-0.2136	0.5302	0.3476	-0.1826
Prompt-rubric sim.	0.7744	0.5000	-0.2744	0.7658	0.4702	-0.2956
FT MiniLM	0.8872	0.7885	-0.0988	0.8829	0.7916	-0.0913
FT DistilBERT	0.9699	0.8750	-0.0949	0.9692	0.8744	-0.0947

5.6 Hard OOD generalisation (honesty probe)

The crash is the point. Rule baselines lose the most: keyword $\Delta \approx -0.21$ accuracy; prompt-rubric $\Delta \approx -0.27$ (to 0.50), which is expected given adversarial keyword traps and tools-vs-rag paraphrases that break cue orderings. FT models degrade more gently but *clearly*: DistilBERT 0.9699 $\rightarrow$ 0.8750; MiniLM 0.8872 $\rightarrow$ 0.7885 (≈ 9 –10 absolute points). A 0.97 in-domain score should not be read as near-solved routing—OOD shows residual confusion on entity-heavy RAG $\rightarrow$ tools and, critically, under-escalation (escalate $\rightarrow$ tools), and supports the suspicion that synthetically expanded v2 labels are partly too easy.

Still above the crashed rubric. On OOD, both FT encoders remain well above the collapsed prompt-rubric (0.50), so the Phase 1 “beat the rubric” criterion still holds under this hardness probe—but with tempered absolute claims.

5.7 Phase 2 hybrid stack simulation

Honesty fence. All cost and latency figures below are arithmetic under `configs/phase2_cost_model.yaml` (fixed unit rates). They are **not** a cloud bill, **not** live LLM API timings, and carry **no** answer-quality EM/F1 or Arena scores. Artefacts: `results/phase2/cost_latency_summary.json`, `rag_hit_at_k.json`, `tools_stub.json`.

Checkpoint disclosure. Phase 2 predictions were made from the DistilBERT checkpoint saved by the OOD suite (trained on `train_v2`, early-stopped on `eval_v2`; `outputs/router_distilbert_v2_for_ood/best`), not from a fresh training run. Its in-domain accuracy is 0.985 (recorded in `results/phase2/phase2_summary.json` under `router_sanity`)—slightly above the Phase 1 single-run point estimate (0.970, Table 5.1), and optimistic because the same split was used as the early-stop monitor. OOD accuracy is identical (0.875) in both. Phase 2 cost tables therefore reflect a mildly favourable in-domain route mix; the OOD table is the fairer read.

Reading the cascade comparison. Under the model, FT DistilBERT routes cost $\approx 0.25\times$ always-RAG+escalate on `eval_v2` (relative mean cost 0.250) and $\approx 0.22\times$ on OOD, with large-call rate 0.218 (ID) / 0.173 (OOD) versus 1.0 for both cascade baselines. p50 latency drops from 2100 ms (always-RAG+escalate) to 365 ms (FT). The cheap **rag** profile

Table 5.4: Phase 2 (**simulation**, not a cloud bill): mean cost, large-call rate, and latency under the fixed unit model (`eval_v2`, $n=133$). Source: `results/phase2/cost_latency_summary.json`.

Policy	Mean cost (sim)	Large-call rate	p50 lat. (ms)	p95 lat. (ms)
Always escalate	1.0000	1.000	1800	1800
Always RAG+escalate	1.1500	1.000	2100	2100
FT DistilBERT (ours)	0.2874	0.218	365	1815
Gold oracle routes	0.3025	0.226	350	1920

Table 5.5: Phase 2 (**simulation**, not a cloud bill): same fixed unit model on hard OOD (`eval_ood`, $n=104$).

Policy	Mean cost (sim)	Large-call rate	p50 lat. (ms)	p95 lat. (ms)
Always escalate	1.0000	1.000	1800	1800
Always RAG+escalate	1.1500	1.000	2100	2100
FT DistilBERT (ours)	0.2558	0.173	365	1815
Gold oracle routes	0.3297	0.212	350	2500

is the **FT+RAG** path (retrieval + small responder, no large call); large+RAG is only the `escalate_large+needs_rag` combination. These are *route-composition* savings under fixed unit rates—not measured dollars.

FT cheaper than gold on OOD (honesty note). On `eval_ood`, FT DistilBERT simulated cost (0.256) is *lower* than gold oracle (0.330). This is not evidence of better routing: the OOD confusion matrix (Section A.4) shows that DistilBERT under-escalates 5/22 `escalate_large` queries ($4 \rightarrow \text{tools}$, $1 \rightarrow \text{rag}$) and mis-routes 5/31 `rag` queries to `tools`. Under the unit cost model, these errors happen to be *cheaper* because tools and rag have lower unit costs than escalate. The simulation therefore flatters FT cost on OOD by converting quality loss (under-escalation) into apparent savings—exactly the kind of risk that live answer-quality metrics (future work) must catch.

Local mini-RAG. BM25 over 19 hand-written company-ish markdown docs; 62 primary-`rag` qrels. Hit@1=0.984, hit@3=1.000, hit@5=1.000 (`results/phase2/rag_hit_at_k.json`). This checks whether the toy corpus covers labelled retrieval intents; it is not a measure of live RAG answer quality.

Tool stubs. On primary=`tools` rows, deterministic `jira_create/github_comment` stubs achieved schema success rate 1.000 on both `eval_v2` and `eval_ood` (`results/phase2/tools_stub.json`). No real Jira/GitHub calls were made.

5.8 Experimental environment

Runs were done in the repository virtualenv with PyTorch and Hugging Face Transformers on CPU (no GPU acceleration was needed at this data scale). Wall-clock time for DistilBERT bootstrap+ k -fold orchestration was in the range of a few minutes on the shared research box

(2026-09-17, Europe/Dublin timestamps in JSON artefacts). Hyperparameters are fixed in YAML; seeds are reported; metrics JSON is the source of truth for tables in this chapter.

5.9 Remaining ablations (future)

True LLM-API prompted 4-way router; remove tools / standalone-small / escalate; knowledge-FT anti-pattern vs behaviour-only FT; natural domain-shift beyond synthetic OOD. Answer-quality EM/F1 remains out of scope until live backends are available, including live answers on the FT+RAG path (small model + retrieved docs). This chapter contains **no invented answer-quality metrics** and **no live LLM API bills**.

Chapter 6

Discussion

6.1 Positioning honesty

Any positive Phase 1 result must be framed as evidence that a small encoder can fit the expanded 4-way labelling policy—not as proof of end-to-end hybrid-system superiority. Claims remain fenced against Adaptive-RAG, QueryRouter, SlimPLM, RouteLLM, and HybridLLM [Jeong et al., 2024, Chen et al., 2025, Tan et al., 2024, Ong et al., 2025, Ding et al., 2024]: what is studied here is the *joint* behavioural space and knowledge≠behaviour training target, without inventing adaptive RAG or cascading. If a concurrent system closes the same 4-way + separation gap, novelty claims shrink to the when-is-FT-worth-it analysis (H6, including future live-prompt comparisons) or stop.

6.2 Reading the Phase 1 numbers

Majority accuracy ≈ 0.31 is the four-way floor under near-balance. Keyword accuracy ≈ 0.58 shows that shallow lexical cues help but miss escalate without overt verbs. The prompt-rubric simulated baseline (≈ 0.77) is the critical non-neural reference: it encodes the same labelling document a prompted LLM would be given, without calling an LLM. FT MiniLM (≈ 0.89 point; bootstrap 0.89 ± 0.03) and DistilBERT (≈ 0.97 ; 0.97 ± 0.02) exceeded that reference on the held-out split. Stratified k -fold means are lower (MiniLM ≈ 0.77 , DistilBERT ≈ 0.91), which is a reminder that a single split can be optimistic—hence both are published.

Possible label simplicity. DistilBERT’s in-domain ceiling is consistent with template-regular workplace labels whose decisive cues (“create Jira”, “our SLA”, “design a plan”) are aligned with both human rationales and model shortcuts. The hard OOD probe was written precisely to stress that hypothesis: ambiguity, multi-intent, tools-vs-rag paraphrases, entity-heavy queries, and keyword traps. Measured drops (DistilBERT ≈ -0.095 acc; prompt-rubric ≈ -0.274) support treating high in-domain scores as *upper bounds under easy labels*, not as proof of robust behavioural understanding. The corrected OOD confusion matrix shows under-escalation (escalate→tools) and rag→tools confusion as the main failure modes—operationally more dangerous than the tools over-prediction pattern seen in-domain. The fact that scores crash on OOD—especially for cue-heavy baselines—is intentional honesty

for a workshop submission.

6.3 When is FT worth it?

Decision criteria are proposed based on **measured Phase 1 deltas**, with Phase 2 simulation supplying illustrative route-composition cost terms:

1. **Beat the strong non-neural rubric.** If FT macro-F1 does not exceed the prompt-rubric simulated baseline by a margin larger than uncertainty (here: MiniLM +0.12 macro-F1, DistilBERT +0.20 on the held-out point estimates; MiniLM k -fold mean is *comparable* to the rubric point score), then shipping the rubric (or a true prompted LLM) should be preferred until more labels or features are available.
2. **Account for training and drift cost.** FT is worth it when (a) the query distribution is stable enough that periodic re-labelling is cheaper than per-query LLM routing calls, and (b) the measured accuracy/F1 lift translates into fewer unnecessary `escalate_large/rag` calls under Phase 2 budgets.
3. **Prefer FT when tools/RAG confusions are costly.** Confusion analysis shows that keyword/rubric errors concentrate on missing `escalate/rag`; FT reduces those misses. If the product risk is *tools over-prediction* (DistilBERT’s residual in-domain pattern) or *under-escalation* (DistilBERT’s OOD pattern), tuning thresholds on the tools class or adding tool-precondition checks is better than abandoning FT altogether.
4. **Prefer prompt/rubric when labels are scarce or shifting fast.** With $n \ll 500$ or weekly policy changes, a documented rubric (or LLM prompt) can be updated faster than a re-trained encoder.
5. **Do not equate simulation with cloud \$/query.** Routing F1 $\neq$ cloud \$/query; Phase 2 simulation supports route-composition savings under a fixed unit model, but live LLM/RAG quality and real bills remain unmeasured.

Phase 1 verdict (narrow, tempered). On this synthetic workplace labelling policy at $n \approx 500/130$, FT DistilBERT clears the rubric bar under bootstrap, k -fold, *and* hard OOD (0.875 vs rubric OOD 0.50); FT MiniLM clears it on held-out and OOD but is closer under k -fold. Absolute DistilBERT in-domain 0.97 must not be oversold: the OOD 0.875 should be reported alongside it. Verdict: “it is worth measuring DistilBERT-class encoders for this policy; claims of near-solved routing are not supported.”

6.4 Threats to validity

Construct validity. Gold labels encode *our* LABELING.md policy, not an external oracle of user utility. High FT accuracy partly means “fits the rubric,” which is necessary but not enough for product quality.

Internal validity. A single primary seed (42) was used for point estimates; this was mitigated by bootstrap and k -fold, not by multi-seed full grids. The prompt-rubric was authored with knowledge of the same guidelines used to label data—this is optimistic for the rubric and conservative for claiming FT novelty over an independent prompt.

External validity. Synthetic English workplace queries were used; no production traffic. Template families (definitions, “create Jira...”, “design a plan...”) may inflate neural scores via lexical regularity—partially checked by `eval_ood`, which is still synthetic and author-constructed, not organic shift.

Statistical conclusion validity. Eval $n=133$ still gives non-trivial CI width (MiniLM accuracy 95% bootstrap CI roughly 0.83–0.93). DistilBERT’s very high point estimate should be read with the k -fold mean (≈ 0.91) and OOD accuracy 0.875 as sobriety checks.

Baseline fairness. No LLM-API prompted router was tested in Phase 1; the rubric simulation is disclosed as such. Keyword is intentionally weak; comparing only to majority would overstate FT gains.

6.5 Failure modes

- **Over-triggering RAG** on parametric questions (adds cost and latency).
- **Under-escalating** hard reasoning that looks lexically simple (keyword/rubric pattern; DistilBERT OOD: 5/22 escalate queries mis-routed to tools/rag).
- **Tools over-prediction** on borderline retrieval or reasoning cases (DistilBERT in-domain residual: 4 off-diagonal errors, all predicted tools).
- **RAG/tools confusion** on ops queries that mention both tickets and “what failed” (MiniLM OOD: 9/31 rag→tools; DistilBERT OOD: 5/31 rag→tools).
- **FT degradation under domain shift** when workplace jargon drifts (H6; natural shift is still future work).
- **Label noise** in synthetic rationales propagating into the classifier.
- **Simulated cost masking quality loss:** under-escalation on OOD makes FT routes cheaper than the gold oracle (Section 5.7), hiding quality risk behind apparent savings.

6.6 Limitations

Labels are synthetic workplace-style examples, not production logs. End-to-end quality depends on RAG corpus quality and tool reliability, which are separate from the behavioural policy claim [Yan et al., 2024, Asai et al., 2023]. Phase 1 does not measure cloud \$/query or downstream EM/F1 (Phase 2 adds simulated cost/latency only). True prompted large routers have not been run yet (only the rubric simulation was used). FT-last polish remains out of scope as a control mechanism (Chapter 3).

6.7 Ethics

Mild safety examples are routed toward refusal-capable answer paths; the dataset must not include actionable harm labelled as tool invocations. Routing mistakes that under-escalate safety-sensitive queries are treated as first-class risks for any live evaluation design.

6.8 Implications of the Phase 2 simulation

The fixed-unit cascade comparison (Section 5.7) shows that an FT DistilBERT policy can cut *simulated* mean cost to roughly one-quarter of always-RAG+escalate while keeping the large-call rate near 0.2, with p50 latency dropping from 2100 ms to 365 ms under the model. Local BM25 hit@ k and tool-stub schema checks confirm that the toy retrieval/tool surfaces are internally consistent—they do **not** prove product-grade RAG or tool reliability. On OOD, the FT policy appears even cheaper than the gold oracle (0.256 vs 0.330), but this is an artefact of under-escalation errors—routing hard queries to cheaper actions—not a genuine routing improvement. This interplay between routing accuracy and simulated cost shows why live answer-quality metrics are needed before system-level cost savings can be claimed.

Still deferred. (i) budget-match FT vs a *true* prompted 4-way LLM router API; (ii) natural domain-shift and freshness splits beyond synthetic OOD; (iii) multi-annotator agreement; (iv) answer-quality metrics from live backends. System tables in this thesis are filled **only** from simulation JSON, never from invented EM/F1 or live API bills.

Chapter 7

Conclusion

In this thesis (v1.1, corrected confusion matrices), an FT-first hybrid behavioural router was studied with a unified 4-way action space (including FT+RAG as the **rag** primary: the small model answers from retrieved docs), knowledge≠behaviour separation, and a when-is-FT-worth-it agenda grounded in measured Phase 1 deltas. No claim is made of inventing adaptive RAG or LLM cascading; the joint niche left open by those established lines of work [Jeong et al., 2024, Chen et al., 2025, Tan et al., 2024, Ong et al., 2025, Ding et al., 2024] is targeted.

Phase 1 takeaway. On in-domain eval ($n=133$), measured baselines establish a transparent ladder: majority accuracy 0.308, keyword 0.579, prompt-rubric simulated 0.774 (not an LLM API). FT MiniLM reached 0.887 (bootstrap 0.886 ± 0.027 ; 5-fold 0.766 ± 0.045) and FT DistilBERT 0.970 (bootstrap 0.969 ± 0.015 ; 5-fold 0.910 ± 0.022). On hard OOD ($n=104$; never trained on), the same DistilBERT dropped to 0.875 and the prompt-rubric to 0.500—evidence that synthetically expanded labels may be partly too easy, reported on purpose. Error analysis shows the residual risk shifting from missed escalate/rag (rules) toward tools over-prediction in-domain and under-escalation on OOD—the latter is operationally dangerous because hard reasoning needs get mis-routed to cheaper actions. The decision criteria in Section 6.3 treat beating the rubric under uncertainty *and* reporting OOD gaps as necessary conditions for honest submission.

Phase 2 takeaway (simulation). Under a documented fixed unit cost/latency model, FT DistilBERT routes achieved simulated mean cost 0.287 on `eval_v2` and 0.256 on `eval_ood`, versus 1.15 for always-RAG+escalate, with large-call rates 0.218 / 0.173. Local BM25 hit@1=0.984 on primary-RAG qrels; tool-stub schema success 1.0. These are **not** cloud bills or live answer-quality scores.

Still ahead. True prompted large 4-way router APIs; live RAG/tools/LLM answer quality; natural shift and freshness. FT-last remains optional presentation polish, not the research core.

Submission stance. This document is a workshop / short-paper style Phase 1 (measured routing) + Phase 2 (simulation) report: honest scope, measured routing tables, simulated system cost tables labelled as such, tempered claims, explicit threats to validity, no fabricated answer-quality metrics, and no live LLM API bills.

Appendix A

Dataset, OOD, and Camera-Ready Checklist

A.1 Dataset statistics (v2 + OOD)

Table A.1: Split sizes and primary-action counts (validated).

Split	n	answer_small	rag	tools	escalate_large
train_v2	526	165	118	121	122
eval_v2	133	41	31	31	30
eval_ood	104	27	31	24	22

Table A.2: Domain counts on train_v2 / eval_v2 / eval_ood.

Domain	train	eval_v2	eval_ood
general_knowledge	145	33	13
company_current	104	27	22
tool_action	120	31	24
multi_hop_planning	119	29	22
ambiguous	23	7	20
safety_mild	15	6	3

A.2 Labeling decision order (summary)

Copied from `data/routing/LABELING.md` for the PDF reader:

1. Is there a tool/API side-effect or live check? $\rightarrow$ `tools`.
2. Are company/time-sensitive facts missing, with no hard-reasoning bottleneck? $\rightarrow$ `rag` (FT+RAG: small FT model answers from retrieved docs).
3. Is there a hard multi-step reasoning/planning bottleneck? $\rightarrow$ `escalate_large` (+ aux flags; `needs_rag=true` is large+RAG).

4. Otherwise $\rightarrow$ `answer_small` (small FT alone, including mild refusal routing).

When facts *and* deep analysis are both needed, `escalate_large` with `needs_rag=true` is preferred (large + RAG, not the small FT+RAG path). Invariants: `rag $\Rightarrow$ needs_rag`; `tools $\Rightarrow$ needs_tools`. Validation is done with `scripts/validate_dataset.py`.

Prompt-rubric tie-break note. The deterministic prompt-rubric baseline (`src/fta_router/baselines.py`) uses scored cues with tie-break `tools > escalate_large > rag > answer_small`; it is disclosed as a simulation, not an LLM API.

A.3 Hard OOD construction (summary)

`eval_ood` ($n=104$) is a held-out hardness probe (not training data): ambiguity, multi-intent, tools-vs-rag paraphrases, entity-heavy queries, and adversarial keyword traps. Rationales carry `[ood:<tag>]` prefixes. Full write-up: `LABELING.md`. Metrics can be reproduced using: `scripts/eval_ood_suite.py  $\rightarrow$  results/ood/`.

A.4 OOD confusion matrices (corrected label order)

Rows = true, columns = predicted; label order `answer_small`, `rag`, `tools`, `escalate_large`. Source: `results/ood/minilm_ood.json`, `distilbert_ood.json` (corrected per Section A.9).

FT MiniLM (`eval_ood`, $n=104$).

<code>answer_small</code> :	25, 0, 1, 1
<code>rag</code> :	2, 20, 9, 0
<code>tools</code> :	1, 1, 22, 0
<code>escalate_large</code> :	0, 0, 7, 15

Dominant failures: `rag $\rightarrow$ tools` (9/31) and `escalate $\rightarrow$ tools` (7/22)—severe under-escalation under hardness.

FT DistilBERT (`eval_ood`, $n=104$).

<code>answer_small</code> :	26, 0, 0, 1
<code>rag</code> :	0, 26, 5, 0
<code>tools</code> :	0, 2, 22, 0
<code>escalate_large</code> :	0, 1, 4, 17

Same shape, milder: `rag $\rightarrow$ tools` (5/31); `escalate` mis-routed on 5/22 (4 $\rightarrow$ tools, 1 $\rightarrow$ rag). The in-domain tools-over-prediction tendency becomes under-escalation on hard queries—the operationally dangerous direction.

A.5 Paraphrase pairs

Twenty-four pairs in `data/routing/paraphrase_pairs_v2.json` share gold `primary_action`. Measured agreement (`results/ood/paraphrase_consistency.json`): keyword 0.750, prompt-rubric 0.917, FT MiniLM 0.875, FT DistilBERT 0.917.

A.6 Result artefact map

- `results/v2/baselines/*.json` — majority, keyword, prompt-rubric (in-domain; Phase 1 measured).
- `results/v2/minilm_v2.json`, `distilbert_v2.json` — in-domain point estimates + CI summaries.
- `results/v2/*_ci.json` — full CI payloads from `scripts/eval_bootstrap_ci.py`.
- `results/ood/*` — OOD metrics, paraphrase consistency, ID→OOD `summary.json`.
- `results/phase2/*` — Phase 2 *simulation* cost/latency, RAG hit@*k*, tool stubs (**not** cloud bills).
- Seed-scale continuity: `results/*_seed.json` (n=38; prior pilot).

A.7 Honesty checklist for readers

- Phase 1 tables contain measured routing metrics only.
- Phase 2 tables contain fixed-unit *simulations*; no live LLM API bills or timings.
- The prompt-rubric baseline is not an LLM API.
- DistilBERT’s high held-out score must be read with *k*-fold mean ≈ 0.91 , OOD accuracy 0.875, and threats in Section 6.4.
- OOD score drops (especially rubric/keyword crashes) are reported intentionally.
- Originality fence: no invention claim for adaptive RAG or cascading.
- No invented EM/F1, Arena, or pretend API answer-quality numbers.
- FT confusion matrices were corrected in v1.1 (label-order erratum, Section A.9); scalar metrics were never affected.
- Phase 2 predictions were made from the OOD-suite checkpoint (in-domain 0.985; disclosed in Section 5.7).

A.8 Camera-ready checklist (v1.0)

Completed for Thesis v1.0 (camera-ready draft) — 17 Sep 2026:

- ☒ Title/abstract match contribution niche; originality fence present in abstract.
- ☒ Front matter: version “Thesis v1.0 (camera-ready draft) — 17 Sep 2026”; author Vijay Kumar.
- ☒ Consistent Phase 1 (measured routing) vs Phase 2 (simulation) language.
- ☒ All numeric tables cite measured or simulation JSON under **results/** (no invented cells).
- ☒ In-domain and OOD tables both present; OOD protocol states “not trained on OOD.”
- ☒ Prompt-rubric disclosed as deterministic simulation (not LLM API).
- ☒ Phase 2 system cost tables filled from **results/phase2/** and labelled as simulation (not cloud bills).
- ☒ Threats to validity include label simplicity / template regularity.
- ☒ Related-work matrix cites Adaptive-RAG, QueryRouter, SlimPLM, RouteLLM, HybridLLM.
- ☒ Dataset validated (`validate_dataset.py`) for train/eval/OOD.
- ☒ Reproducibility commands in README / **results/*/README.md**.
- ☒ PDF builds via `make -C thesis pdf / latexmk`.
- ☒ Ethics: mild safety never labelled as **tools**; no actionable harm.
- ☒ Author checklist: no overclaim of E2E hybrid-system superiority from Phase 1 alone; no live API bill claims from Phase 2 simulation.

A.9 Erratum: confusion matrix label ordering (v1.1)

Affects: FT MiniLM and FT DistilBERT confusion matrices in v1.0 (Chapter 5 error analysis, **results/v2/*.json**, **results/ood/*.json**).

Root cause: `sklearn.preprocessing.LabelEncoder.fit()` sorts classes alphabetically (**answer_small**, **escalate_large**, **rag**, **tools**), but the evaluation scripts decoded integer indices using the **PRIMARY_ACTIONS** tuple order (**answer_small**, **rag**, **tools**, **escalate_large**). Both **y_true** and **y_pred** were decoded with the same wrong mapping, so **accuracy**, **macro-F1**, **bootstrap CI**, **k-fold**, and **paraphrase agreement** are all correct—only confusion matrix row/column labels and the error analysis narrative were affected.

Impact: In v1.0, DistilBERT’s residual in-domain errors were attributed to “escalate over-predictions” when the correct pattern is *tools over-predictions* (all 4 off-diagonal errors

predict tools). On OOD, v1.0 described “over-escalation under hardness” when the actual dominant failure is *under-escalation* (escalate→tools: 4/22) and rag→tools confusion (5/31).

Not affected: Phase 2 simulation predictions were decoded correctly—`run_phase2_sim.py` already used `LabelEncoder` alphabetical inverse mapping (see its `predict_with_checkpoint` docstring and the `label_decode` note in `results/phase2/phase2_summary.json`)—so all Phase 2 cost, latency, RAG, and tool-stub figures stand as published.

Fix: Confusion matrices in Chapter 5 and Section A.4 have been corrected to the true label ordering. Scripts (`train_router_local.py`, `eval_bootstrap_ci.py`, `eval_ood_suite.py`) now use `le.inverse_transform()` for decoding. Committed artefacts under `results/` were permuted into the declared label order (an accuracy-invariant transform, validated against gold class counts) and carry a `label_order_erratum` field; `ft_*` prediction strings in `results/ood/paraphrase_consistency.json` details were remapped to true classes (agreement values unchanged).

Bibliography

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511*, 2023. URL <https://arxiv.org/abs/2310.11511>.
- Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023. URL <https://arxiv.org/abs/2305.05176>.
- Shufan Chen, He Zheng, and Lei Cui. When and how to augment your input: Question routing helps balance the accuracy and efficiency of large language models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 3621–3634, Albuquerque, New Mexico, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-naacl.200. URL <https://aclanthology.org/2025.findings-naacl.200/>. QueryRouter framework.
- Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=02f3mUtqnM>. Also arXiv:2404.14618.
- Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*, 2024. URL <https://arxiv.org/abs/2403.12031>.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.389. URL <https://aclanthology.org/2024.naacl-long.389/>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In

- Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Aman Madaan, Pranjal Aggarwal, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems*, 2024. Also arXiv:2310.12963.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=8sSqNntaWX>. Also arXiv:2406.18665.
- Yuanzhe Shen, Yide Liu, Zisu Huang, Ruicheng Yin, Xiaoqing Zheng, and Xuanjing Huang. SATER: A self-aware and token-efficient approach to routing and cascading. *arXiv preprint arXiv:2510.05164*, 2025. URL <https://arxiv.org/abs/2510.05164>. EMNLP 2025.
- Jiejun Tan, Zhicheng Dou, Yutao Zhu, Peidong Guo, Kun Fang, and Ji-Rong Wen. Small models, big insights: Leveraging slim proxy models to decide when and what to retrieve for LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4420–4436, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.242. URL <https://aclanthology.org/2024.acl-long.242/>. SlimPLM; also arXiv:2402.12052.
- Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. CRAG—corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024. URL <https://arxiv.org/abs/2401.15884>.
- Woongyeong Yeo, Kangsan Kim, Soyeong Jeong, Jinheon Baek, and Sung Ju Hwang. UniversalRAG: Retrieval-augmented generation over corpora of diverse modalities and granularities. *arXiv preprint arXiv:2504.20734*, 2025. URL <https://arxiv.org/abs/2504.20734>.
- Tian Yu, Shaolei Zhang, and Yang Feng. Auto-RAG: Autonomous retrieval-augmented generation for large language models. *arXiv preprint arXiv:2411.19443*, 2024. URL <https://arxiv.org/abs/2411.19443>.
- Jiarui Zhang, Xiangyu Liu, Yong Hu, Chaoyue Niu, Fan Wu, and Guihai Chen. RAGRouter: Learning to route queries to multiple retrieval-augmented language models. *arXiv preprint arXiv:2505.23052*, 2025. URL <https://arxiv.org/abs/2505.23052>.